
MemMux: Runtime Verification and Honest Resource Attribution for Fleets of Parallel Coding Agents

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Developers increasingly run not one coding agent but a fleet of them side
2 by side on a single workstation. The tools they reach for—terminal multi-
3 plexers like `tmux` and a new generation of agent managers—were built to
4 arrange windows, not to govern memory. When ten agents each spawn
5 language servers, test runners, and browsers, no standard tool in that
6 stack can say how much memory belongs to which agent, guarantee that
7 a terminated agent’s descendants are actually gone, notice a child that
8 has escaped its agent, or keep the machine off the swap cliff without
9 an OOM kill silently discarding uncommitted work. We argue these are
10 *runtime-verification* problems: an agent-hosting substrate should continu-
11 ously emit observable signals that an operator—or an auditor—can monitor
12 and check while agents run in production. We present MEMMUX, a local
13 runtime that treats resource governance as a set of checkable guarantees—
14 honest per-agent attribution, complete reclamation, escaped-process visibil-
15 ity, bounded footprint under overcommit, and low monitoring overhead—
16 and a claims-disciplined benchmark that measures each guarantee against
17 `tmux`, a purpose-built agent multiplexer, and a raw-process baseline run-
18 ning identical workloads. Under a binding memory budget on a Linux
19 host, MEMMUX holds the fleet *under* budget (7.5 GiB) with zero swap—
20 admitting a subset and reclaiming under pressure—while the ungoverned
21 tools run every agent, pin the machine at its RAM ceiling (2× over bud-
22 get), and spill ~2 GiB into swap; it reclaims 100% of a terminated agent’s
23 process subtree where the raw baseline strands half of it (ten orphaned pro-
24 cesses, 303 MiB); and it is the only system that surfaces an escaped child,
25 detecting 10/10 injected escapes versus a capability the others structurally
26 lack. We report an honest cost: the always-on 1 Hz attribution scan stays
27 near 0.6% CPU for a single agent but reaches 2.7% at ten, over our 2%
28 target. Driving the same harness with real Claude Code sessions confirms
29 that 100% attribution and low overhead carry over from the stub to live,
30 multi-process agent trees. We release the engine, the benchmark, and a
31 one-command reproducer.

1 Introduction

33 A year ago, running an autonomous coding agent meant babysitting a single session. Today
34 it is normal to fan a task out across several agents at once—one refactoring, one writing tests,
35 one chasing a flaky build—and to watch them in a grid of panes. The agents themselves have

improved quickly [4, 10, 12, 13]; the substrate they run on has not. Developers arrange these fleets with terminal multiplexers such as `tmux` [8] or with purpose-built agent managers, and both were designed to place windows on a screen, not to answer questions about memory.

Those questions matter because a coding agent is not a single process. It is a tree: the model’s CLI, a shell, language servers, test and build workers, sometimes a headless browser. A handful of such trees can exhaust a laptop’s RAM. When they do, the operating system’s response—the Linux OOM killer, or swap thrashing—is blunt and invisible: a process is killed mid-write, or the whole machine grinds, and the agent’s uncommitted edits are simply gone. Nothing in the stack was accountable for that memory in the first place. `tmux` does not know that pane 7’s language server belongs to the same logical agent as pane 3’s test runner; it cannot tell you that a double-forked child has escaped supervision; and it has no budget it is trying to keep.

We think the right lens for this is *runtime verification* [1]: rather than prove properties of the substrate ahead of time, instrument it so the properties it claims can be *checked while it runs*, and surface any violation rather than hide it. Cluster schedulers made an analogous move a decade ago—Borg combined admission control, over-commitment, and process-level isolation, and treated monitoring as a first-class output [11]. We bring that stance down to a single developer machine and to the specific failure modes of agent fleets.

This reframes an infrastructure problem as a verification one, on an axis the agent-evaluation literature largely leaves open. Most work on verifying agents asks whether an agent’s *outputs* are trustworthy—did it resolve the issue, is the reward genuine, is the behaviour aligned. We ask a complementary question that only sharpens once agents run in parallel and unattended: does the *substrate that hosts them* uphold checkable operational guarantees—honest accounting, complete reclamation, no silently lost work—that an operator can observe while the fleet runs? These are always-on signals over the live execution, not one-shot scores, and a fleet whose host cannot answer them is unverified no matter how capable each agent is.

This paper contributes: (i) **five verification signals for agent fleets** (Section 4)—honest per-agent attribution, complete reclamation on teardown, escaped-process visibility, bounded footprint (and preserved work) under overcommit, and bounded monitoring overhead—each a monitor with a stated threshold; (ii) **MemMux** (Section 3), a local runtime that enforces them; and (iii) **a claims-disciplined benchmark** (Section 4) that drives `tmux`, an agent multiplexer, and a raw baseline through *identical* deterministic workloads, tags every process as agent or manager overhead, and never reports a number it did not measure—where a competitor cannot be driven honestly, it is recorded as skipped, not estimated. On a 32 GB Linux reference host the guarantees hold and, under overcommit, separate MEMMUX from the ungoverned tools by $\sim 5\times$ at ten agents (Section 5); we are candid about the scope—a deterministic-stub workload, modest fleets ($N \leq 10$), and an overhead target not yet met—in Section 6.

2 Background and Related Work

Language-model agents. Interleaving reasoning with tool use [13] and adding self-reflection [10] turned language models into agents that take long sequences of actions. Coding is the canonical proving ground: SWE-bench frames the task as resolving real GitHub issues against a repository [4], and SWE-agent shows a well-designed agent–computer interface matters as much as the model [12]. Multi-agent settings, first studied for open-ended behavior [9], are now a practical reality on developer machines. This work is orthogonal to the agents: we govern whatever they spawn.

Terminal multiplexers. `tmux` [8] is the default way developers run many long-lived sessions at once. It is excellent at persistent panes and detach/reattach—the honest baseline any “run many agents” tool must beat—but a pane is just a process group under a server with no notion of a logical agent, a memory budget, or a process that has wandered out of its subtree.

87 **Resource management.** Cluster managers solved fleet-scale governance long ago: Borg
88 with admission control and over-commitment [11], Mesos with two-level scheduling [3], and
89 Kubernetes as the open-source lineage [2]. The enforcement primitives they use—Linux con-
90 trol groups [5] and the OOM killer [6]—live in every modern kernel. MEMMUX borrows the
91 ideas (reserve, admit, reclaim before you thrash) but targets a single un-orchestrated lap-
92 top, and it deliberately avoids hard cgroup containment in favor of signal-based supervision.
93 This is an explicit trade-off: cgroups give hard, kernel-enforced limits but need privilege
94 and per-machine setup, whereas signal-based supervision runs unprivileged and portably
95 (macOS included) at the cost of best-effort rather than hard containment—a child can, for
96 an instant, escape before reconciliation catches it (H3).

97 **Memory accounting.** Summing resident set size (RSS) over a process tree double-counts
98 shared pages. The proportional set size (PSS) exposed through `/proc/<pid>/smaps_rollup`
99 divides each shared page by its sharers and is the honest per-process figure [7]; macOS offers
100 `phys_footprint` as its analogue. MEMMUX attributes by PSS (RSS fallback) so a task’s
101 number reflects memory it is responsible for.

102 **Runtime verification.** Rather than verify a system statically, runtime verification instru-
103 ments it and checks a specification against the execution trace, reacting to violations [1].
104 Our five signals are exactly such monitors: cheap, always-on checks with explicit thresholds
105 whose failures are events, not silence.

106 3 The Design of MemMux

107 MEMMUX is a Rust workspace: a daemon (`memmuxd`) that owns state and does the governing,
108 and a terminal client that attaches over a Unix domain socket. A *task* is a durable, logical
109 agent; a *runtime* is one physical launch of a provider inside a pseudo-terminal. The design
110 is organized around making each signal checkable (Figure 1).

111 **Attribution.** Once per second the daemon snapshots the host process tree and, for each
112 running task’s provider root, sums the PSS of the root and all descendants. That per-task
113 figure feeds the budget, the client’s task view, and—because we record which pids were seen
114 under each root—the escaped-process check. Attribution is scoped to processes MEMMUX
115 launched, so unrelated host activity never pollutes a task’s number.

116 **Recursive reclamation.** When a task is terminated (by the operator or because its
117 provider exited), MEMMUX reaps the *entire* owned subtree. The order is load-bearing: the
118 subtree is signalled while the root is alive, because once the root dies its children reparent to
119 `init` and the tree that identifies them is lost. For a provider that already exited, MEMMUX
120 reaps from the pids recorded while the task ran. The target is $\geq 99.5\%$ of owned descendants
121 gone within ten seconds.

122 **Escaped-process reconciliation.** A child that double-forks and reparents to `init` has
123 escaped its agent—the classic way a leak hides. Each sample, MEMMUX reconciles the
124 pids seen under a task against the live tree; a once-owned pid now living outside every task
125 subtree is flagged *escaped* and surfaced as an event. Reconciliation is scoped to the managed
126 set, so the signal is a real “we lost track of a process we started,” not host noise.

127 **Admission and the pressure ladder.** The daemon holds a memory budget (physical
128 RAM minus reserves for the OS, interactive apps, and a swap-safety margin). Starting
129 a task reserves its predicted peak; if the reservation would exceed the budget the task
130 stays queued with a visible reason rather than over-subscribing the machine. When live
131 usage climbs anyway, a graduated *pressure ladder* responds before the machine thrashes:
132 reclaim escaped children, hibernate the lowest-priority idle task, recycle the largest provider,
133 and—only at the top, and only after persisting a checkpoint of the victim’s Git state—
134 terminate the lowest-priority tree. Preserving work always precedes destroying a process.
135 Every governance action writes an audit decision with its reason and evidence, which is
136 what makes the system verifiable rather than merely automatic.

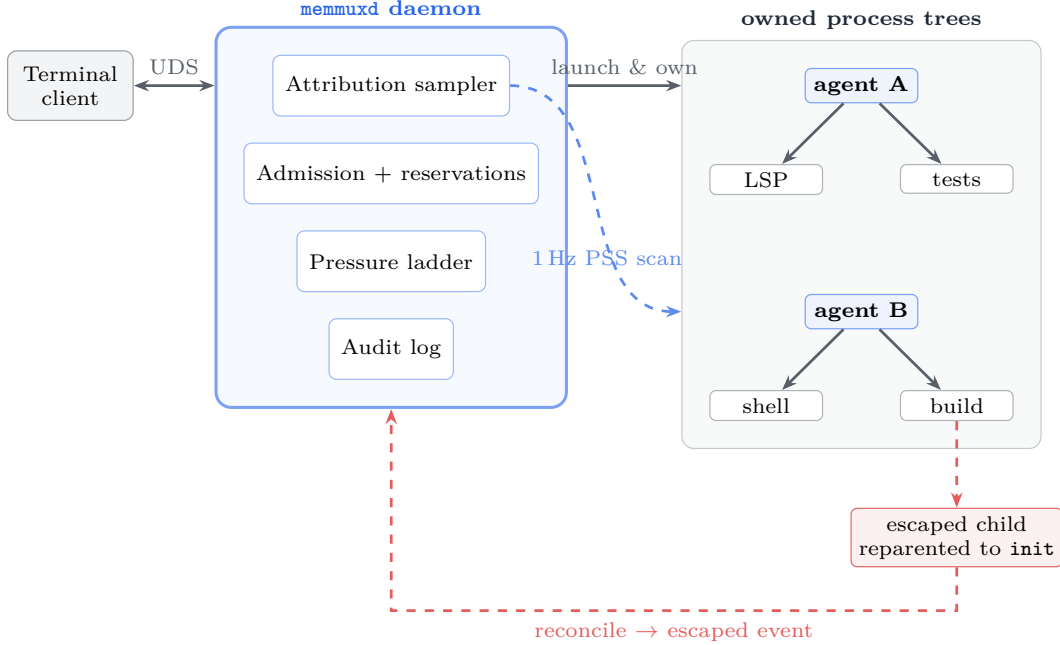


Figure 1: MEMMUX architecture. A terminal client drives the `memmuxd` daemon over a Unix domain socket. The daemon launches and *owns* each agent’s process tree; a 1 Hz sampler sums proportional memory (PSS) across every owned subtree (H1), reconciliation flags any child that double-forks out to `init` (H3), and admission, the pressure ladder, and an audit log turn the budget into checkable, logged decisions (H2/H4/H5).

137 4 Verification Signals and Methodology

138 We frame governance as runtime monitors over the fleet’s execution. The observed *trace*
139 is the sequence of once-per-second process-tree samples and lifecycle events; each signal
140 is an *invariant* over that trace with an explicit threshold (Table 1), and a breach is sur-
141 faced as a violation event rather than silently tolerated. Four are properties of the running
142 system (H1–H4); H5 is the *cost* of monitoring, not an invariant, and H4 bundles three sub-
143 properties—admission, bounded footprint, and preservation of uncommitted work—which
144 we report separately below. MEMMUX both *observes* these signals and *enforces* them (ad-
145 mission, reclamation); the baselines can only be observed. These are empirical monitors
146 with measured pass/fail behaviour, not machine-checked proofs, so we claim the signals are
147 checkable and checked, not that the engine is sound and complete in the formal sense. A
148 benchmark harness drives every launcher through identical, deterministic workloads and
149 evaluates the monitors from one external process sampler—an *oracle* using the daemon’s
150 own accounting code, pointed at whatever root pids a launcher reports, so measurement is
151 product-agnostic.

152 **Launchers.** Each launcher starts N identical agents and reports the pids that constitute
153 the fleet: a *raw* baseline (spawn the stub directly), `tmux` (one pane per agent on an isolated
154 server), and MEMMUX (create N tasks through the real daemon). A fourth, the `herdr` agent
155 multiplexer, is driven through its socket API where installed; on our stock Linux reference
156 host it was absent and is reported as skipped rather than guessed, as are `dmux` (which
157 requires a controlling terminal and cannot be driven headlessly), `cmux`, and `agentmux`.

158 **Workload.** A deterministic stub agent replays a recording of `allocate` / `spawn-child` / `emit`
159 / `sleep` steps, touching every page it allocates so its resident memory is real. Using a stub
160 rather than a live model is a deliberate choice: it makes runs reproducible and identical
161 across launchers, at the cost of realism (Section 6). A *hold* scenario keeps N agents, each

	Signal	Monitor / threshold
H1	Attribution	$\geq 95\%$ of a launched tree’s proportional memory mapped to its owning task
H2	Cleanup	$\geq 99.5\%$ of an owned subtree gone ≤ 10 s after the launcher’s normal teardown
H3	Escaped visibility	every process that escapes its agent is surfaced as an event
H4	Bounded / no-loss	footprint held near budget under overcommit; un-committed work checkpointed before any reclaim
H5	Overhead	continuous monitoring costs $\leq 2\%$ CPU at fleet scale

Table 1: The five runtime-verification signals and their pass thresholds.

with a live child, concurrently resident for the measurement window; without it, short workloads finish before a fleet can be observed—an error we hit and corrected early.

Claims discipline. The harness records each launcher’s version, the measurement date, and the host; it tags every sampled process as agent or manager overhead so the two are reported separately; and it emits a number only for a monitor it actually measured. A launcher that cannot be driven contributes “unsupported,” never a fabricated zero. These rules are as much a contribution as the engine: fair comparison of agent-hosting tools has no accepted methodology, and an unfair one is worse than none. One consequence deserves care. Because the oracle can attribute *any* launcher’s tree after the fact, per-agent attribution is *measurable* for all of them—the real-agent run reports 100% for every launcher (Table 4). H1 as a signal is the stricter question of whether a launcher exposes that attribution *natively and continuously* to an operator, which only MEMMUX does; **tmux** and the raw baseline surface no such figure themselves. It is that native capability, not the oracle’s coverage, that Table 2 marks.

5 Evaluation

Setup. Signals H1–H3 and H5 run on an AWS **m7i.2xlarge** reference host (8 vCPUs, Intel Xeon Platinum 8488C, ≈ 31 GiB RAM, Ubuntu 24.04), sweeping $N \in \{1, 5, 10\}$ over two trials. The bounded-footprint claim (H4) needs *real* memory pressure, so we run it on a deliberately smaller 15.3 GiB host (**m7i.xlarge**, 4 vCPUs, 12 GiB swap) with a *realistic* per-agent footprint: each agent holds ~ 1.4 GiB, matching the Standard-class reservation prior, so admission is well-calibrated (reservation $\approx$ actual) rather than over-reserving, and $N=12$ agents (~ 18 GiB) genuinely overcommit the machine. That run is *five* trials; we report mean $\pm 95\%$ CI throughout (at the two-trial sweeps the interval is indicative). A macOS cross-check (Apple M4 Max, 32 GiB) that also exercised **herdr** agreed qualitatively: like the other ungoverned managers, **herdr** reaped its subtrees on teardown but saw no escapes and grew with the fleet. Table 2 summarises the head-to-head; the subsections that follow are the evidence behind each cell.

5.1 H4: bounded footprint under overcommit

We make the budget *binding*. Each agent holds a realistic ~ 1.4 GiB (the Standard-class reservation prior, so admission is well-calibrated rather than over-reserving a tiny stub), and $N=12$ such agents (~ 18 GiB) overcommit a 15.3 GiB host. MEMMUX is given a 7500 MiB budget—roughly half what the fleet wants. Table 3 reports peak total footprint and peak swap over five trials. The ungoverned launchers run all twelve agents: their resident set pins the machine at its ~ 15 GiB RAM ceiling— $2.0\times$ over the budget—and spills 1.9–2.2 GiB into swap. MEMMUX instead governs: its admission planner defers seven starts and its pressure ladder performs twelve reclamations, holding peak footprint at 7479 MiB (*under* budget), settling to 4.5 GiB steady, with **zero** swap.

The honest trade-off is throughput: MEMMUX keeps ~ 5 agents resident and defers the rest, so enforcing the budget means not running every agent at once. But that is the *point* of a

Launcher	Attrib.	Cleanup	Escaped	Footprint (budget)	Swap	Ckpt.
raw baseline	×	50%	×	15075 MiB	1882 MiB	×
tmux 3.4	×	100%	×	15347 MiB	2165 MiB	×
MEMMUX 0.9.0	✓	100%	✓ 10/10	7479 MiB	0	✓ [†]

Table 2: Head-to-head on the signals (Linux reference host). MEMMUX is the only launcher that natively attributes per-agent memory (H1), surfaces escaped processes (H3), holds the fleet *under* a binding memory budget with zero swap (H4), and can checkpoint dirty work before a reclaim. It *matches* **tmux** on teardown cleanup (H2). Footprint/swap are peaks under a binding 7500 MiB budget on a 15.3 GiB host ($N=12$); the ungoverned tools overcommit to the RAM ceiling and spill ~ 2 GiB into swap. Attribution is the *native* capability, not the oracle’s coverage (§4); MEMMUX also pays a monitoring cost the baselines do not (H5, §5). [†]Checkpoint is confirmed by a daemon-level unit test, not triggered end-to-end. Each cell is the headline result for that signal.

Launcher	Peak (MiB)	Steady (MiB)	Peak swap (MiB)
raw baseline	15075	15069	1882
tmux 3.4	15347	15069	2165
MEMMUX 0.9.0	7479	4512	0

Table 3: Binding-budget *overcommit* ($N=12$ agents, ~ 1.4 GiB each, 15.3 GiB host, five trials). MEMMUX’s 7500 MiB budget is half the ~ 15 GiB the ungoverned fleet uses; MEMMUX holds peak footprint under budget with zero swap (7 deferrals + 12 reclamations), while raw and **tmux** overcommit the machine $2\times$ and spill ~ 2 GiB into swap.

budget—the ungoverned tools “succeed” at running all twelve only by driving the machine into swap, where an OOM kill can strike. The *no-lost-work* half of H4 is the safety net for exactly that case: before the pressure ladder’s *emergency* stage terminates a victim it first captures a durable checkpoint (git HEAD + patch hash). Here the twelve reclamations were hibernate/recycle actions that never reached emergency termination, so this path was not exercised end-to-end; a daemon-level test confirms it directly—a task with a dirty worktree, forced into the emergency stage, produces a checkpoint carrying its patch hash *before* the process is terminated. Ungoverned launchers have no checkpoint mechanism at all.

5.2 H2: complete cleanup on teardown

Each launcher starts a fleet whose agents each hold a memory-bearing child, then is torn down exactly as it normally would be (raw: kill the roots; **tmux**: `kill-server`; MEMMUX: terminate each task). Ten seconds later we count survivors from the owned set. The raw baseline strands every child it spawned—its teardown kills only the roots, orphaning the grandchildren—and the leak grows with the fleet: one process and 30.6 MiB at $N=1$, ten processes and 303 MiB at $N=10$ (50% reclaimed at every size). **tmux** and MEMMUX both reap their full subtree at every size (0 leaked, 100% reclaimed). The honest reading, which the claims discipline exists to surface: on *explicit* teardown, cleanup separates MEMMUX from the raw baseline, not from a real multiplexer—**tmux** reaps its subtrees too.

5.3 H3: escaped-process visibility

We inject escapes: a stub forks an intermediate that spawns a memory-holding grandchild and then exits, reparenting the grandchild to `init` while it stays alive. The harness independently confirms the reparenting, then asks each launcher what it detected. MEMMUX surfaces every escape—1/1, 5/5, 10/10—through events read from the daemon; the baselines have no mechanism to notice, so they are recorded as unsupported, never scored as zero-of-measured. This is a capability, not a tuning difference, and it is the cleanest separation from a real competitor. Detection is bounded by the sample interval: reconciliation runs at 1 Hz, so a child that both escapes *and* exits within a second could slip between two samples. Our injected escapes hold memory for several seconds—comfortably longer

than the interval—and shortening the interval trades detection latency for overhead (H5); a persistent leak, which is the case that matters, is caught on the next sample regardless.

5.4 H1 & H5: what the attribution scan delivers, and what it costs

We pair these two signals because they are the benefit and the price of a single mechanism: the always-on 1 Hz PSS scan produces the attribution guarantee (H1) and incurs the monitoring cost (H5). Across every run MEMMUX attributed 100% of each launched tree’s proportional memory to its owning task *natively*, and the external oracle independently confirmed the figure (H1). The cost of governance in *memory* is small and we report it plainly: the MEMMUX daemon’s own resident footprint is 4.6–5.4 MiB (growing slightly with the fleet), against 2.4 MiB for the `tmux` server and zero for the raw baseline. The cost in *CPU* is the honest weak spot (H5): the MEMMUX daemon’s total CPU—dominated by the always-on 1 Hz full-tree PSS scan, though we did not ablate the two—is 0.6% at a single agent, 1.5% at five, and 2.7% at ten, over our 2% target once the fleet reaches ten. The scan is $O(\text{processes})$ and currently re-reads the whole tree each tick; incremental sampling that touches only changed pids is the obvious fix and is future work. We state the miss rather than bury it.

5.5 Validation on real coding agents

To test whether the attribution and overhead results are artifacts of the stub, we ran the *same* harness against a real coding agent. Each “agent” is a headless Claude Code session (`claude -p`, v2.1.170) editing a scratch Git repository, launched through the identical sampling and teardown pipeline via the benchmark’s `--agent-cmd` mode. We report an $N=3$ hold run on the macOS host (Apple M4 Max, 36 GiB). A live Claude Code session is a multi-process, Node-based tree, so this is a strictly harder attribution target than the single-process stub.

Table 4 gives the result. MEMMUX attributes 100% of every real agent tree to its owning task—the H1 gate holds on live agents, not just the stub—and the manager cost stays small (7.2 MiB resident, 0.012% CPU at three agents, consistent with the small-fleet end of the H5 curve). Three concurrent real sessions occupy ~ 0.85 GB (~ 290 MiB each), confirming that the pressure the system governs is real and per-agent-substantial. Attribution and overhead thus generalize from the stub to real, multi-process agent trees.

Cleanup on live agents. That hold run let every session finish, so all launchers reclaimed cleanly. To check the cleanup *mechanism* on *live* agents we ran a second experiment (Linux, `m7i.xlarge`) in which three real Claude Code agents (v2.1.197) are *forcibly torn down mid-work*, while their `node` process trees are still resident. The raw baseline strands **three real processes and 525 MiB** of live agent memory (50% reclaimed)—killing the root `node` orphans its children to `init`, exactly as with the stub—while both `tmux` and MEMMUX reap the full subtree (0 leaked, 100%). Consistent with H2, this is a *raw*-versus-governed result, not a separation from `tmux`; its value is that the cleanup behaviour the stub isolates reproduces on real agent trees, and that the naive “run it in a terminal” baseline strands half a gigabyte of live agent memory. The escaped-process condition (H3) did not arise here, as Claude Code does not double-fork out of supervision.

6 Discussion and Limitations

Synthetic workload. The largest threat to validity: agents are a deterministic memory-ballast stub, not live models. We trade realism for reproducibility and for identical workloads across launchers, which is what makes the comparison fair. The *qualitative* guarantees—*attribution*, *cleanup*, *escape visibility*, *admission*—do not depend on the workload, but the specific megabytes for a real Claude- or GPT-driven session will differ. We take a first step in §5.5, running real Claude Code sessions through the same harness: 100% attribution and low overhead hold on live, multi-process agent trees, and forcibly killing them mid-work reproduces the raw baseline’s cleanup leak (525 MiB) that the stub isolates. A larger live-

Table 4: Real Claude Code agents ($N=3$, hold scenario, macOS) driven through the identical harness. Every launcher attributes the full real (Node-based) agent tree; MEMMUX’s manager overhead is small. Footprint is steady-state provider+manager memory; at $N=3$ without an overcommit budget the totals are comparable across launchers—this run isolates *attribution and overhead on real trees*, not the overcommit footprint differentiator of Table 3.

Launcher	Total steady (MiB)	Manager (MiB)	Tree attributed
raw baseline	883.6	0.0	100%
tmux 3.6a	883.6	2.4	100%
herdr 0.8.0	929.6	18.2	100%
MEMMUX 0.8.0	843.5	7.2	100%

agent study—more agents and longer sessions, and reproducing the H4 overcommit and H3 escape conditions on live trees—remains the most valuable next step.

One host, modest fleets. We evaluate a single 32 GB Linux host (with a macOS cross-check) and $N \leq 10$. The harness sweeps to $N=20$ and the reproducer emits the full curve; we simply did not run larger fleets, on which the CPU-overhead question (H5) becomes sharper.

Static reservations. MEMMUX reserves each task’s *class-predicted* peak—a static prior. Our H4 run sizes agents to that prior on purpose, so admission is well-calibrated; but for an atypically small agent the prior over-reserves (deferring starts that would have fit), and for an atypically large one it under-reserves. An exponentially-weighted predictor that tightens the reservation toward each task’s *observed* peak is designed and is future work; the bounded-footprint guarantee is unaffected—only the admit count moves with the predictor’s accuracy.

Overhead not yet met. Monitoring exceeds 2% CPU by ten agents (above); incremental sampling is the planned fix. As stated in §4, these are empirical monitors, not machine-checked proofs—checkable and checked, not formally verified.

7 Reproducibility

MEMMUX and its benchmark are open source (MIT).¹ A single command runs the full matrix—every launcher, scenario, and fleet size, with repeated trials—captures the host specification, and regenerates every table and figure here plus the raw per-sample and per-process time series. Competitor versions and the measurement date are embedded in the generated report, and any launcher that could not be driven is listed with its reason. Reproducing the evaluation on another host is one invocation. The same harness accepts a `--agent-cmd` flag that substitutes a real coding-agent command (e.g. `claude -p ...`) for the stub, so the real-agent validation of §5.5 is reproducible through the identical pipeline; that run’s report and raw per-process series are archived under `paper/realagent-run/`.

Each run captures a host specification, a Markdown report, and per-cell time series (RSS, PSS, USS, swap, and page faults per process on Linux) plus the figures. We withhold the exact invocation and the repository here to preserve review anonymity; the engine, the benchmark, and the one-command reproducer—including the binding-budget H4 configuration used for Table 3—are released with the code.

8 Conclusion

Fleets of coding agents have outrun the tools we use to run them: multiplexers arrange them, but nothing governs them. We argued that memory governance for such fleets is

¹Code, benchmark, and a one-command reproducer are open-source and will be released; the repository URL is withheld to preserve double-blind review.

a verification problem and built MEMMUX to make five guarantees checkable at runtime, with a benchmark honest enough to admit where those guarantees do *not* distinguish us. The payoff is concrete: under a binding budget MEMMUX holds the fleet under budget and *off swap* where the ungoverned tools overcommit the machine ($2\times$ over budget, ~ 2 GiB into swap), it reclaims a terminated agent’s whole subtree where the raw baseline strands half a fleet, and it gives full visibility into escaped processes that the alternatives cannot see—alongside an honest overhead cost we have not yet driven under 2%. The path from here is a live-agent workload and larger fleets; the reproducer makes both a single command away. Verifying an agent is not only checking what it produced—it is also checking that the system running it kept its promises; the signals and the claims discipline here are a step toward the latter.

References

- [1] Ezio Bartocci, Yliès Falcone, Adrian Francalanza, and Giles Reger. Introduction to runtime verification. In *Lectures on Runtime Verification: Introductory and Advanced Topics*, volume 10457 of *Lecture Notes in Computer Science*, pages 1–33. Springer, 2018. doi: 10.1007/978-3-319-75632-5_1.
- [2] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, Omega, and Kubernetes. *Communications of the ACM*, 59(5):50–57, 2016. doi: 10.1145/2890784.
- [3] Benjamin Hindman, Andy Konwinski, Matei Zaharia, Ali Ghodsi, Anthony D. Joseph, Randy Katz, Scott Shenker, and Ion Stoica. Mesos: A platform for fine-grained resource sharing in the data center. In *Proceedings of the 8th USENIX Conference on Networked Systems Design and Implementation (NSDI)*, 2011.
- [4] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.
- [5] Linux Kernel Documentation. Control group v2. <https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v2.html>, 2024. Accessed 2026-08.
- [6] Linux Kernel Documentation. Out of memory management. <https://www.kernel.org/doc/html/latest/admin-guide/mm/concepts.html>, 2024. Accessed 2026-08.
- [7] Linux Kernel Documentation. The /proc filesystem: smaps and smaps_rollup (proportional set size). <https://www.kernel.org/doc/html/latest/filesystems/proc.html>, 2024. Accessed 2026-08.
- [8] Nicholas Marriott and tmux contributors. tmux: a terminal multiplexer. <https://github.com/tmux/tmux>, 2024. Accessed 2026-08.
- [9] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023. arXiv:2304.03442.
- [10] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.11366.
- [11] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at Google with Borg. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys)*, 2015. doi: 10.1145/2741948.2741964.
- [12] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.
- [13] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.